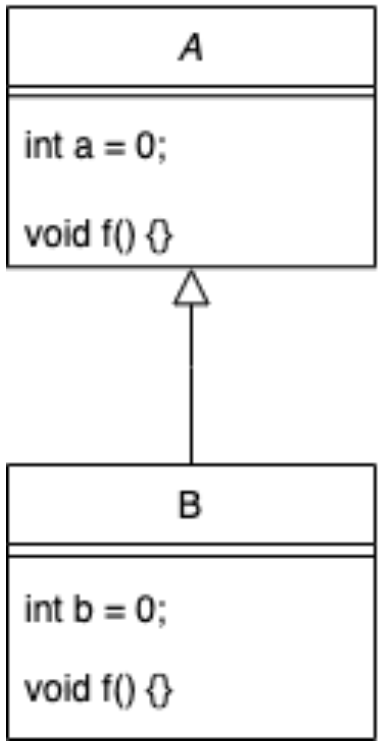
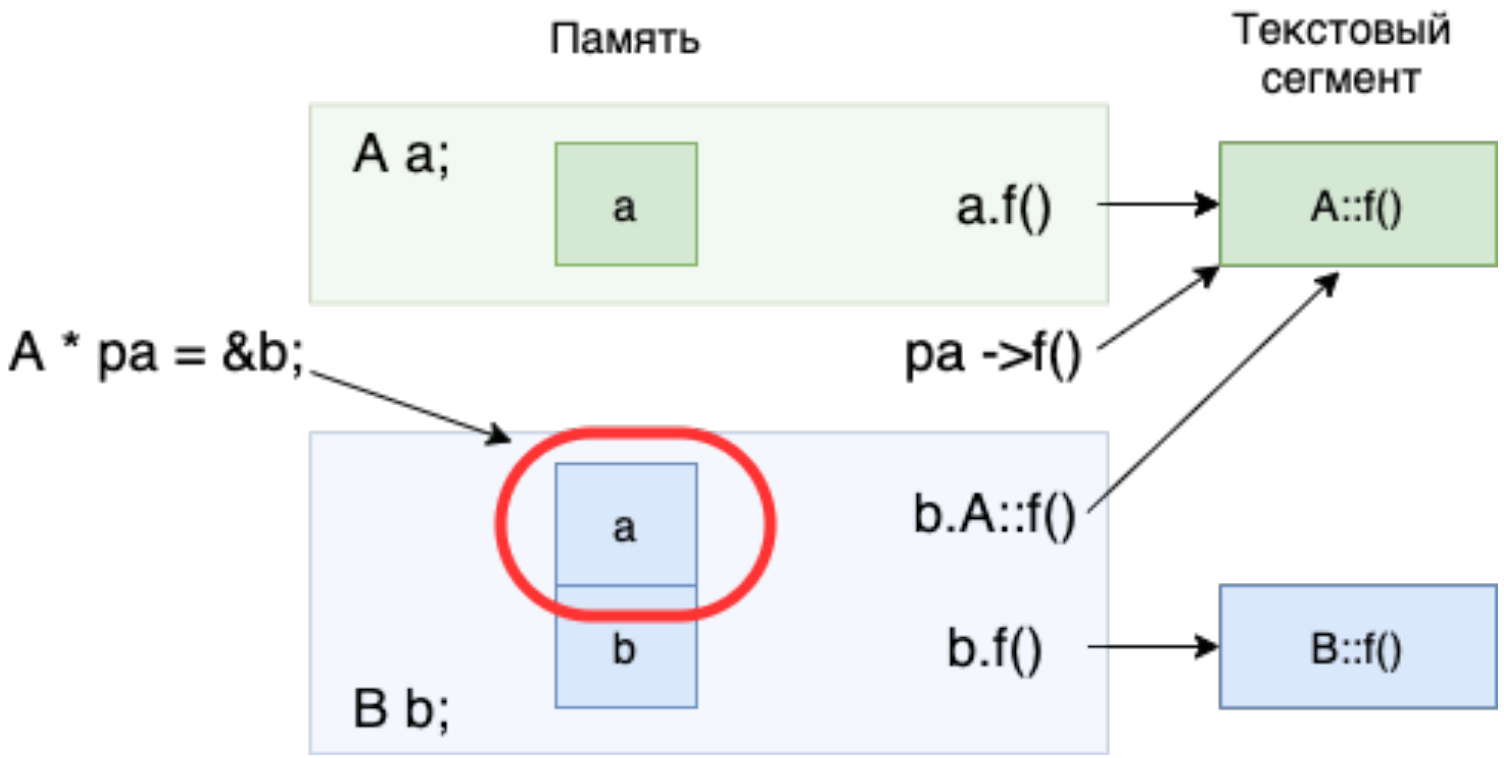
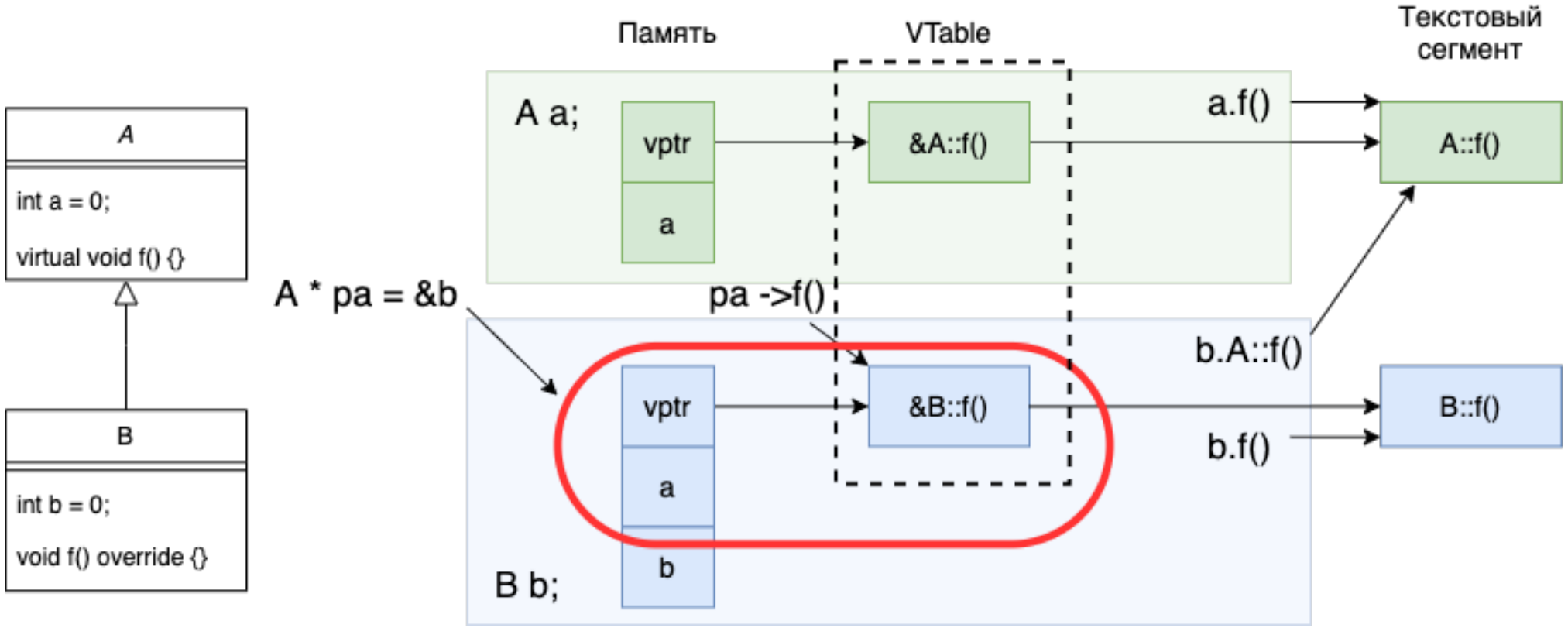
Полиморфизм

Простое наследование

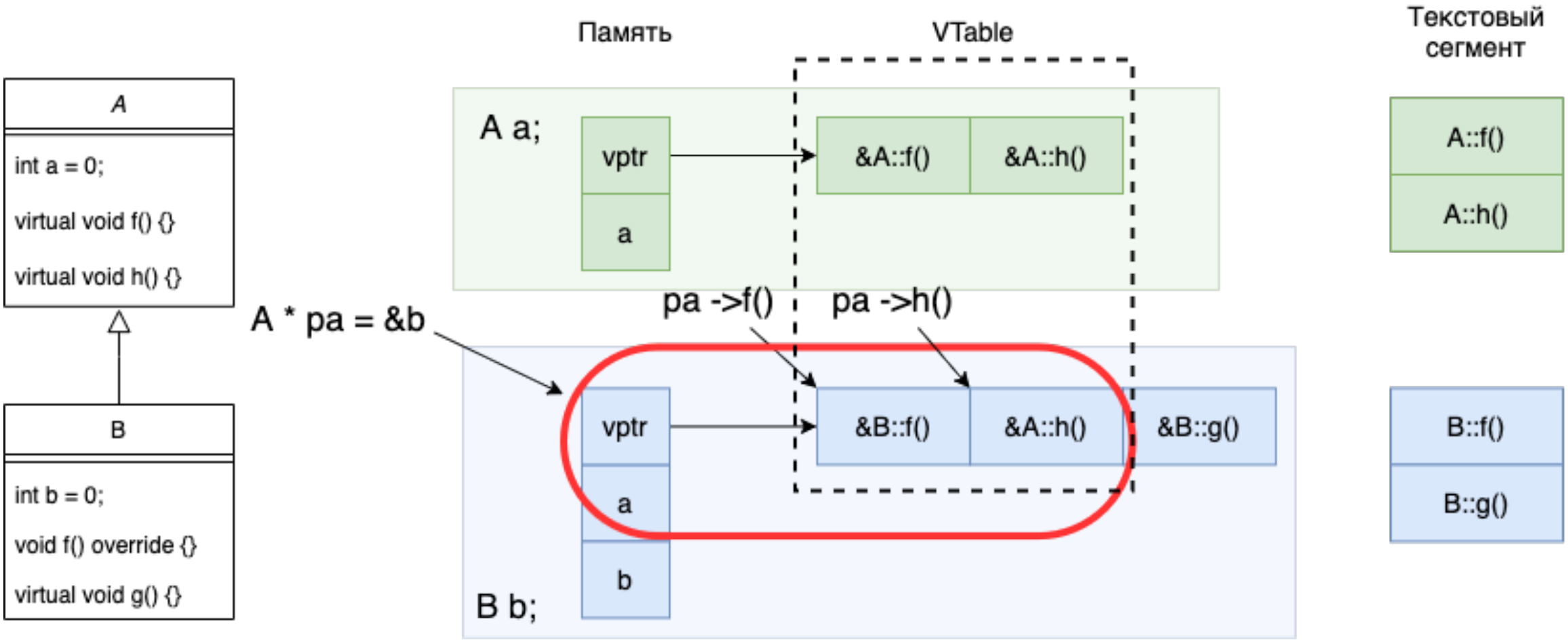




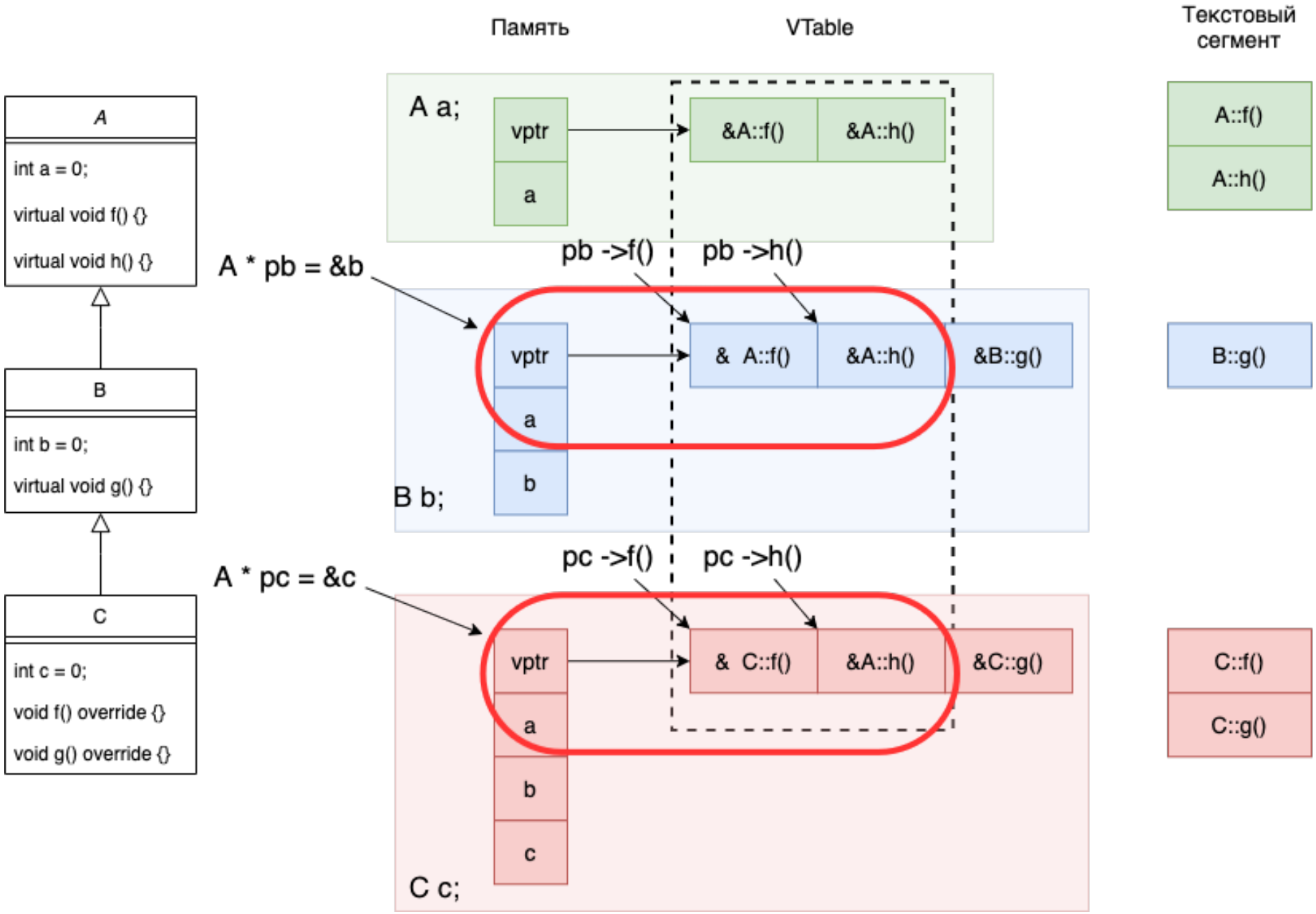
Виртуальные функции



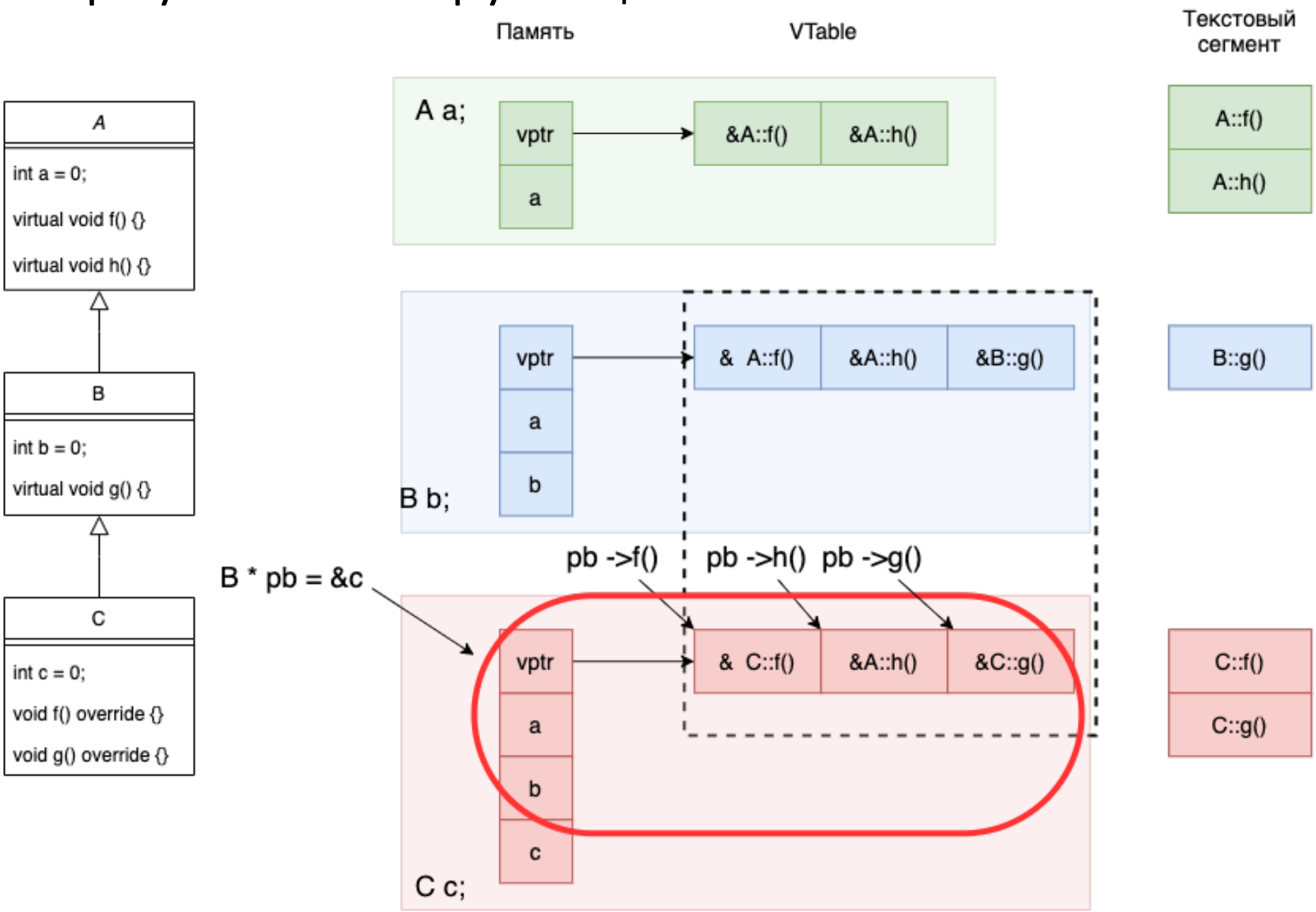
Виртуальные функции



Виртуальные функции



Виртуальные функции



?

```
class A {
public:
    A() = default;
    virtual void print() const { std::cout << "print() from A" << std::endl; }
};
class B: public A {
public:
    B() : A() {}
private:
    void print() const { std::cout << "print() from B" << std::endl; }
};


int main() {
    B b;
    A& a = b;
    a.print();
}
```

1. Компилятор смотрит на тип объекта a — это A&. В классе A метод print() объявлен public, значит, вызов разрешён.

2. Поскольку метод виртуальный, в процессе выполнения вызывается версия, соответствующая типу объекта — B::print(). То, что в B этот метод private, уже не имеет значения

Красивый подсмотренный пример

```
class A {
public:
    A() = default;
    virtual void print() const { std::cout << "print() from A" << std::endl; }
};
class B: public A {
public:
    B() : A() {}
private:
    void print() const { std::cout << "print() from B" << std::endl; }
};
int main() {
    A a;
    B b;

    char c = '0';
    std::cin >> c;

    A& pa = (c == 'y'? a : b);
    pa.print();
}
```

Виртуальный деструктор. Постановка задачи

```
class A {
public:
    A() = default;
    void print() const { std::cout << "print() from A" << std::endl; }
    ~A() { std::cout << "Dtor A" << std::endl; }
};
class B: public A {
    int * b = new int;
public:
    B() : A() {}
    B(int n): b(new int[n]) {}
    void print() const { std::cout << "print() from B" << std::endl; }
    ~B() {delete [] b; std::cout << "Dtor B" << std::endl; }
};
int main() {
    A* a = new B(5);
    a->print();  // print() from A
    delete a;
}
```

```
print() from A
Dtor A
```

Виртуальный деструктор. Почему так

Происходит это потому, что удаление производится через указатель на базовый класс и для вызова деструктора **компилятор использует раннее связывание**. Деструктор базового класса не может вызвать деструктор производного, потому что он о нем ничего не знает. В итоге, часть памяти, выделенная под производный класс, безвозвратно теряется.

Чтобы этого избежать, деструктор в базовом классе должен быть объявлен как виртуальный.

Виртуальный деструктор. Решение

В решении проблемы вызов деструктора используется **позднее связывание**, то есть при разрушении объекта берется указатель на класс, затем из таблицы виртуальных функций определяется адрес нужного нам деструктора, а это деструктор производного класса, который после своей работы, как и полагается, вызывает деструктор базового.

```
class A {
public:
    A() = default;
    void print() const { std::cout << "print() from A" << std::endl; }
    virtual ~A() { std::cout << "Dtor A" << std::endl; }
};
class B: public A {
    int * b = new int;
public:
    B() : A() {}
    B(int n): b(new int[n]) {}
    void print() const { std::cout << "print() from B" << std::endl; }
    ~B() {delete [] b; std::cout << "Dtor B" << std::endl; }
};
```

Абстрактный класс

Существуют классы, в которых виртуальные функции не имеют реализации. Такие функции называются **чисто виртуальными**, а классы, в которых они находятся, – **абстрактными**.

Можно сказать, что это интерфейсный класс, то есть описывает поведение объекта, но не определяет его.

Нельзя создать экземпляр абстрактного класса.

Чисто виртуальная функция

```
class Shape {
public:
    virtual void print () const = 0;
    virtual ~Shape() = 0;   //Определять чистый виртуальный деструктор обязательно
};
class Triangle: public Shape {
public:
    double A = 60;
    double B = 60;
    double C = 60;
    Triangle() {}
    void print() const override {
        std::cout << "Углы: " << A << ' ' << B << ' ' << C << std::endl; }
};
class Rectangle: public Shape {
public:
    double x = 0;
    double y = 0;
    void print() const override { std::cout << "Стороны: " << x << ' ' << y << std::endl; }
};
```

Объект абстрактного класса создать нельзя

```
class Shape {
public:
    virtual void print () const = 0;
    virtual ~Shape() {}
};
class Triangle: public Shape {
public:
    double A = 60;
    double B = 60;
    double C = 60;
    Triangle() {}
    void print() const override { std::cout << "Углы: " << A << ' ' << B << ' ' << C << std::endl; }
};
class Rectangle: public Shape {
public:
    double x = 0;
    double y = 0;
    void print() const override { std::cout << "Стороны: " << x << ' ' << y << std::endl; }
};

int main() {
    Rectangle rec;
    Shape& sh = rec;
    sh.print();
}
```

Класс-наследник, не определяющий чисто виртуальную функцию, тоже будет считаться абстрактным

```
class A {
public:
    int a = 2;
    virtual void f(int x = 1) {std::cout << x << "A\n";}
    virtual void h() = 0;
};
class B: public A  {
public:
    int b = 2;
    void f (int x = 2) override {std::cout << x << "B\n";}
};
int main () {
    B b;                                   2 ⊗ Variable type 'B' is an abstract class
    A *pa  = &b;
    pa->f();
}
```

```cpp
class Shape {
public:
    virtual void print () const = 0;
    virtual ~Shape() {}
};
class Triangle: public Shape {
    double A = 60; double B = 60; double C = 60;
public:
    Triangle(double A, double B, double C):A(A), B(B), C(C) {}
    void print() const override {
        std::cout << "Углы: ” << A << ‘ ‘ << B << ' ' << C << std::endl;
    }
};
class Rectangle: public Shape {
    double x = 0;
    double y = 0;
public:
    Rectangle(double x, double y): x(x), y(y) {}
    void print() const override {
        std::cout << "Стороны: "
        << x << ' ' << y << std::endl;
    }
};
```

Полиморфизм. PS:Наличие виртуального деструктора обязательно.

```cpp
int main() {
    Shape* sh[3];
    sh[0] = new Triangle(30,90,60);
    sh[1] = new Rectangle(3,6);
    sh[2] = new Rectangle(4,4);
    for (int i = 0; i < 3; i++) {
        sh[i]->print();
    }
    for (int i = 0; i <3 ; i++) {
        delete sh[i];
    }
}
```

Полиморфизм

Это возможность заменять поведение функций базового класса в классах-наследниках.

Есть ещё специальный полиморфизм (перегрузка процедур и функций, перегрузка операторов)

и параметрический полиморфизм (обобщённая функция, обобщённое программирование – читай, шаблоны)

Определение чисто виртуальной функции

```
class A {
public:
    A() = default;
    virtual void print() const = 0;
};


void A::print () const {
    std::cout << "print() from A" << std::endl;
}
class B: public A {
 public:
  B() : A() {}
  void print() const { std::cout << "print() from B" << std::endl; }
};

int main() {
    B b;
    b.A::print(); //print() from A
}
```

Чисто виртуальный деструктор

1. Необходимо предоставить тело функции для чисто виртуального деструктора
2. В отличие от остальных чисто виртуальных функций, предоставлять определение чисто виртуального деструктора в производном классе *не обязательно*
3. Чисто виртуальный деструктор используется, обычно, для предотвращения создания объекта данного класса

Порядок создания полиморфного объекта

Когда создаётся полиморфный объект:

1. Сначала создаётся его базовая часть, vptr инициализируется указателем на таблицу vtable базового класса
2. Инициализируются поля базового класса
3. Выполняется тело конструктора базового класса
4. vptr переставляется на таблицу vtable производного класса
5. Инициализируются поля производного класса
6. Выполняется тело конструктора производного класса

Уничтожение объекта происходит в обратном порядке

Виртуальная функция не может быть static

Виртуальные статические методы не имеют смысла, поскольку static означает принадлежность к классу, а не к конкретной сущности этого класса.

```
class A {
public:
    int a = 2;
    static virtual void f(){std::cout << "A\n";}
    virtual void h(){std::cout << "Ah\n";}
};
class B: public A  {
public:
    int b = 2;
    virtual void f(){std::cout << "B\n";}
    virtual void h(){std::cout << "Bh\n";}
};

int main () {
    |
}
```

'virtual' can only appear on non-static member functions

Виртуальная функция может быть inline

Ключевое **inline** слово предполагает, что компилятор заменяет код в определении функции вместо каждого вызова этой функции.

```
class A {
public:
    int a = 2;
    inline virtual void f(){ std::cout << "A\n";}
};
class B: public A  {
public:
    int b = 2;
    inline virtual void f() final {std::cout << "B\n";}
};
int main () {
    B b;
    b.f();
    A a;
    a.f();
    A* pa = &a;
    pa->f();
}
```

Таким образом, функция будет «встроена» в код только для вызова функции как метода класса, то есть когда она вызывается как «невиртуальная»

Виртуальная функция должна быть определена

Виртуальные функции нельзя оставлять без определения, кроме ситуации, когда это чистая виртуальная функция.

Компилятор должен сформировать Vtable, то есть положить в виртуальную таблицу адрес вызываемой функции. Но невозможно взять адрес у функции, у которой нет определения.

```
class A {
public:
    int a = 2;
    virtual void f();
};
class B: public A  {
public:
    int b = 2;
    virtual void f() final {std::cout << "B\n";}
};
int main () {
    B b;
    b.f();
}
```

```
  NOTE: a missing vtable usually means the first non-inline virtual member
function has no definition.
```

Вызов виртуальной функции из **конструктора и деструктора** происходит как вызов обычной функции

Поэтому чисто виртуальную функцию вызывать нельзя, даже если она где-то определена

```
class A {
public:
    A() {                   // Call to pure virtual member function 'f' has undefined behavior;
        f();                //overrides of 'f' in subclasses are not available in the constructor of 'A'
    }
    int a = 2;
    virtual void f() = 0;
};
void A::f() {
    std::cout << "Hello";
}
class B: public A  {
public:
    int b = 2;
    void f() override {std::cout << "B\n";}
};
int main () {
    B b;
}
```

```
Pure virtual function called!
```

Чисто виртуальная функция — это **обещание**, что производный класс предоставит реализацию. В момент конструирования базового класса это обещание ещё не выполнено, поэтому вызов такой функции считается ошибкой.

Значения по умолчанию в виртуальных функциях

```
class A {
public:
    int a = 2;
    virtual void f(int x = 1) {std::cout << x;}
};


class B: public A  {
public:
    int b = 2;
    void f (int x = 2) override {std::cout << x;}
};
int main () {
    A* pa = new B;
    pa ->f();  //?
    delete pa;
}
```

Если в родительской и производной виртуальных функциях переданы значения по умолчанию, в дочернюю будет передано значение по умолчанию родителя. Потому что значение по умолчанию устанавливается на моменте компиляции, а дочерняя функция вызывается в момент выполнения

dynamic_cast

Когда dynamic_cast используется для приведения к конкретному типу, возвращаемое значение представляет собой указатель на этот тип только в том случае, если такое преобразование допустимо и успешно выполнено. В противном случае возвращается ноль —признак того, что целевой тип был задан неправильно.

```
class Pet {
    public:
    virtual ~Pet(){}};
class Dog : public Pet {};
class Cat : public Pet {};
int main () {
    Pet* b = new Dog;
    Dog* dl = dynamic_cast<Dog*>(b);    //0x1007461d0
    Cat* d2 = dynamic_cast<Cat*>(b);    //0x0
    Pet* d3 = dynamic_cast<Pet*>(b);    //0x1007461d0
    std::cout << "dl = " << dl << std::endl;
    std::cout << "d2 = " << d2 << std::endl;
    std::cout << "d3 = " << d3 << std::endl;
}
```

для определения фактического типа dynamic_cast использует информацию, хранящуюся в таблице виртуальных функций.

```cpp
struct Animal {
    int a = 2;
    virtual ~Animal() {}

};
struct Dog : public Animal {
    int b = 3;
    void bark() {std::cout << "Gav";}
    ~Dog(){}
};

int main() {
    Animal* a = new Dog();
    // Безопасное преобразование
    Dog* d = dynamic_cast<Dog*>(a);
    if (d) {
        d->bark();
        std::cout << d->b;
    }
}
```

Чтобы dynamic_cast работал корректно:

- **Классы должны быть полиморфными** — содержать хотя бы одну виртуальную функцию (можно просто деструктор)
- **Иерархия должна быть публичной** — открытое наследование

```cpp
int main() {
    Animal a;
    Animal* pa = &a;
    // Теперь преобразование опасное, вернётся 0
    Dog* d = dynamic_cast<Dog*>(a);
    if (d) {
        d->bark();
        std::cout << d->b;
    }
}
```

Когда использовать:

- Необходимо привести указатель от базового к производному
- Нет уверенности в реальном типе объекта

Стратегии наследования

- Производный класс имеет **доступ к защищенным членам базового** (но только для объектов собственного типа);
- **Защищенные данные** приводят к проблемам сопровождения;
- **Защищенные функции** - хороший способ задания операций для использования в производных классах;
- **Открытое наследование** делает производный класс **подтипом базового**;
- **Защищенное** и **закрытое наследование** используются для выражения деталей реализации;
- **Защищенные базовые классы** полезны в иерархиях с дальнейшим наследованием;
- **Закрытые базовые классы** полезны для "ужесточения интерфейса".